**Professores:**

Celso Giusti

Daniel Manoel Filho

Felipe Augusto Stevanim Gregate

Marlon Palata Fanger Rodrigues

# Gerenciamento de Estado

SENAI

# Capacidades Técnicas Abordadas

3. Interpretar os requisitos do sistema, tendo em vista a elaboração dos componentes em ambiente mobile

4. Definir os elementos de entrada, processamento e saída para a codificação das funcionalidades mobile

Introdução

# O que é estado? (State)

Estado é qualquer dado que, quando muda, precisa fazer a interface reagir e se atualizar.

Quando um componente é atualizado e tem um valor alterado, significa que seu estado mudou.

Imaginem um componente que representa um placar de jogo, assim que um time pontua o estado desse componente é alterado para exibir a nova pontuação do time.

Estados

# Arquitetura de Gerenciamento

Nós temos diversas formas de gerenciar o estado de uma aplicação, dependendo de quantos componenentes e telas você quer afetar com a alteração.

- **Estado Local:** Pertence a um único componente. Ex: Um botão de "mostrar/ocultar senha". Controlado via useState.
- **Estado Global:** Dados compartilhados por todo o app. Ex: Tema (Dark/Light), dados do usuário logado.
- **Estado de Servidor:** Dados que vêm de fora (API/Banco de Dados). Lida com cache, loading e erros. Ferramentas: React Query, SWR.

Hooks

# O que são Hooks?

Hooks são funções que permitem que componentes funcionais se conectem ao sistema interno do React. Esse sistema interno controla duas coisas: **estado e ciclo de vida.**

Nesta aula iremos abordar:

- useState
- useRef
- useEffect

Hooks

# useState

O **useState** adiciona uma variável de estado. Chamar a função de atualização causa a re-renderização da tela.

**Como funciona:** Ele te devolve duas coisas: a variável que guarda o valor atual e uma função para atualizar esse valor.

**Regra de ouro**: Sempre que a função de atualização é chamada, o React Native "desenha" (re-renderiza) a tela novamente com a nova informação.

Hooks

# Exemplo useState

```
const [contador, setContador] = useState(0);
return (
  <View style={styles.container}>
    <Text>{contador}</Text>
    <TouchableOpacity onPress={() => setContador(contador + 1)}>
      <Text>Clique para acrescentar o valor</Text>
    </TouchableOpacity>
  </View>
);
```

Hooks

# Dica Bônus

É necessário ter atenção quando o novo valor depende do valor atual do estado, pois em alguns casos isso pode causar bug.

No slide anterior usamos setContador(contador + 1) passando o valor diretamente. Para garantir que o valor mais recente sempre seja considerado, usamos a forma funcional:

Ex: setContador((valorAtual) => valorAtual + 1)

A comunidade costuma fazer: setContador((prev) => prev + 1)

Hooks

# Regras para utilização

O React não executa seus Hooks de forma aleatória.

Internamente, ele mantém uma **lista ordenada** de todos os Hooks do componente — e depende que essa lista seja **sempre a mesma** a cada renderização.

Se a ordem mudar entre uma renderização e outra, o React perde o controle de qual Hook é qual e o comportamento do componente se torna imprevisível.

As Golden Rules existem para **garantir essa ordem**. São duas regras simples que, se seguidas, evitam a maioria dos bugs relacionados a Hooks.

Hooks

# Golden Rule 1

**Chame os Hooks apenas no "Top Level"**: Chame Hooks sempre na raiz do componente, na mesma ordem toda vez. O React depende dessa ordem para funcionar.

```
// ❌ ERRADO — Hook dentro de loop
function Componente() {
  for (let i = 0; i < 3; i++) {
    const [item, setItem] = useState(''); // quebra a ordem
  }
  return <View />
}

// ✅ CERTO — Hooks sempre na raiz, antes do return
function Componente() {
  const [nome, setNome] = useState('');
  const [ativo, setAtivo] = useState(false);

  return <View />
```

Hooks

# Golden Rule 2

**Chame os Hooks apenas de dentro de Funções do React:** Chame Hooks apenas dentro de componentes funcionais React ou dentro de Custom Hooks — nunca em funções utilitárias comuns.

```
// ❌ ERRADO — Hook dentro de função utilitária comum
function calcularIdade() {
  const [idade, setIdade] = useState(0); // não é um componente React
  return idade;
}

// ✅ CERTO — Hook dentro de componente funcional
function Perfil() {
  const [idade, setIdade] = useState(0);
  return <Text>{idade}</Text>
}
```

Hooks

# useRef

O **useRef** também guarda informações, como o useState, mas com uma diferença crucial: ele não redesenha a tela quando o valor muda.

É usado para dois casos: guardar valores internos sem causar re-render, e armazenar referências diretas a elementos da interface (alterar foco em campos de texto).

```
const inputRef = useRef(null)
// ...
<TextInput ref={inputRef} />
<Button onPress={() => inputRef.current.focus()} />
```

Hooks

# useEffect

Executa código com efeitos colaterais fora do fluxo de renderização. O React renderiza o componente primeiro e só depois executa o useEffect

Ele pode ser usado para consumir APIs, buscar dados externos etc.

Você diz ao useEffect: "Ei, depois que a tela estiver pronta, vá buscar esses dados pra mim."

Hooks

# Como funciona: useEffect

**Como funciona:** Ele recebe uma função (o que fazer) e uma lista de dependências (quando fazer).

**O truque da lista:** Se a lista estiver vazia [ ], **ele executa só uma vez quando a tela abre**.

Se tiver uma variável lá dentro [contador], **ele executa toda vez que essa variável mudar**.

Sem o array **ele executa toda vez que a tela é renderizada**.

```
useEffect(() => {
  if (moedas === 5) {
    Alert.alert("Sucesso!", "Você desbloqueou o baú de tesouro! 🎁");
  }
}, [moedas]); // <-- Sempre que 'moedas' mudar, o useEffect acorda e checa
```

Exercício Prático

# Componente de Recados

Você vai criar um componente onde o usuário digita uma mensagem e clica em Enviar para exibi-la na tela.

1. O usuário digita no campo de texto
2. Ao clicar em **Enviar**, a mensagem aparece abaixo
3. O campo de texto é limpo após o envio

Use o useState. para armazenar e atualizar o estado

## ESCOLA SENAI ÍTALO BOLOGNA

Av. Goiás, 139

**Telefone**

(11) 2396-1999

**Instagram**

@senai.itu

**Facebook**

/senaisp.itu

**Site**

https://sp.senai.br/unidade/itu/